

Octalysis Architecture

Proposito

Este documento aterriza la estructura tecnica de la expansion de gamificacion para Riovoley, separando ownership de datos, motores internos y orden de evolucion del feature.

Ownership

- **physical-tests**: dato fisico fuente
- **attendance**: dato de asistencia fuente
- **payments**: dato financiero fuente
- **auth-session**: dato de ingreso/login fuente
- **gamification**: proyeccion motivacional derivada

Motores internos

XP Engine

- calcula XP por fuente verificable
- genera ledger detallado
- separa progreso de recompensas cosmeticas

Streak Engine

- racha mensual
- racha de dias habiles seguidos
- rachas combinadas

Achievement Engine

- logros permanentes
- logros competitivos
- logros secretos
- logros combinados

Challenge Engine

- retos mensuales
- mini-retos semanales
- pre-retos del siguiente ciclo
- campañas especiales

Campaign Engine

- ventanas semanales, mensuales y flash
- catalogo persistido de campañas activas
- snapshot de progreso temporal por estudiante

- recompensas visibles por ventana limitada

Competition Engine

- leaderboards por categoria
- leaderboards por medicion
- records actuales
- rivales a superar

Recommendation Engine

- recomendaciones fisicas orientativas
- recomendaciones segun retos y competencia
- nudges de continuidad
- rutas estrategicas con una principal y hasta dos alternativas
- cadena exacta de accion -> recompensa inmediata -> impacto posterior

Athlete Stage Engine

- interpreta el progreso real como etapas narrativas del atleta
- combina nivel minimo con evidencia de tests, asistencias, pagos y logros
- exige presencia competitiva en etapas altas cuando aplique
- persiste snapshot actual e historial de ascensos

Identity Engine

- apodos
- titulos equipados
- proyeccion publica de identidad
- estado implementado actual:
 - `gamification.titles_catalog`
 - `gamification.student_identity`
 - seleccion de titulo desbloqueado desde el panel del estudiante
 - seleccion persistida de `avatar_style`
 - avatar generado por URL deterministica con DiceBear
 - expansion aprobada:
 - separar `avatar_style` de `avatar_model_slug`
 - catalogo fijo de modelos por estilo
 - modelos base y modelos bloqueados visibles
 - imagen principal unificada avatar/foto
 - portraito compartido entre panel, sidebar y leaderboards
 - estado implementado actual:
 - `avatar_model_slug` persistido
 - opciones disponibles y bloqueadas proyectadas al agregado
 - render del avatar sensible a `style + model`

Economy Engine

- wallet

- ledger de moneda
- catalogo de cosmeticos
- inventario
- equipamiento
- estado implementado actual:
 - `gamification.currency_wallets`
 - `gamification.currency_ledger`
 - extracto visible de monedas en el panel del estudiante
 - `gamification.cosmetic_items_catalog`
 - `gamification.student_cosmetic_items`
 - `gamification.student_cosmetic_equipment`
 - compras/equipamiento via funciones SQL seguras
 - expansion aprobada:
 - todos los cosmeticos deben causar un cambio visible
 - algunos cosmeticos afectan internamente el avatar cuando el modo sea `avatar`
 - para foto de perfil los cosmeticos solo afectan la envoltura exterior
 - estado implementado actual:
 - renderer con fallback visual por slug para nuevos marcos, fondos, insignias y efectos
 - catalogo ampliado por migracion para sumar variedad visible sin nuevo codigo por item

Achievement Visibility Layer

- logros desbloqueados
- logros bloqueados visibles
- logros secretos con pista
- estado objetivo:
 - mantener sorpresa sin ocultar por completo el espacio de descubrimiento
 - soportar progreso visible cuando aplique
- estado implementado actual:
 - `secretAchievements` separado del resto de bloqueados
 - grid propia para secretos con pista y progreso parcial

Surprise Engine

- recompensas ocultas derivadas por combinacion real
- snapshot persistido de descubrimiento por estudiante
- pistas separadas de los logros secretos clasicos
- sin azar artificial en esta fase

Structured Reinforcement Layer

- cierre progresivo de core drivers aun mas debiles
- primera fase persistida:
 - `gamification.athlete_stages_catalog`
 - `gamification.student_current_stage`
 - `gamification.student_stage_history`
- segunda fase persistida:
 - `gamification.campaigns_catalog`

- `gamification.student_campaign_progress`
- tercera fase persistida:
 - `gamification.hidden_rewards_catalog`
 - `gamification.student_hidden_rewards`
- fases siguientes previstas:
 - rutas estrategicas mas editoriales si hiciera falta

Entidades nuevas sugeridas

- `gamification_xp_ledger`
- `gamification_currency_wallets`
- `gamification_currency_ledger`
- `gamification_titles_catalog`
- `gamification_student_titles`
- `gamification_avatar_items_catalog`
- `gamification_student_avatar_items`
- `gamification_student_equipped_items`
- `gamification_student_identity`
- `gamification_login_rewards`
- `gamification_seasonal_campaigns`
- `gamification_student_streaks`
- `gamification_athlete_stages_catalog`
- `gamification_student_current_stage`
- `gamification_student_stage_history`
- `gamification_campaigns_catalog`
- `gamification_student_campaign_progress`
- `gamification_hidden_rewards_catalog`
- `gamification_student_hidden_rewards`

Reglas estructurales

- `XP = progreso`
- `Moneda = personalizacion`
- el login nunca es fuente principal de progresion
- la competencia debe convivir con superacion personal
- cada core driver debe tener:
 - una mecanica visible
 - una evidencia tecnica
 - una prueba automatizada
 - una metrica de producto

Orden de implementacion

1. ledger XP + login diario + racha habil
2. catalogo ampliado de logros y retos
3. apodos + titulos
4. wallet + ledger de moneda
5. avatar configurable + equipamiento

6. tienda cosmetica
7. etapas del atleta persistidas
8. campañas y temporadas
9. vista operativa de entrenador/admin